

计算机学院实验报告

实验题目：八数码问题		学号： 202200130048
日期：2025.10.9	班级： 22.6	姓名： 陈静雯
实验目的： 熟悉和掌握启发式搜索的定义、估价函数和算法过程，并利用启发式搜索算法求解八数码难题，理解求解流程和搜索顺序。		
实验环境介绍： Windows; Python 3.9; Anaconda		
解决问题的主要思路： 1. 启发式搜索算法流程图 <pre>graph TD Start([开始]) --> AddInit[将初始节点加入开放列表] AddInit --> While[While 开放列表不为空:] While --> GetMin[从开放列表取出 f 值最小的节点] GetMin --> IsGoal{如果该节点是目标状态: 是} IsGoal -- 是 --> ReturnPath[返回解路径] IsGoal -- 否 --> AddClosed[将当前节点加入关闭列表] AddClosed --> GenNeighbors[生成所有可能的邻居节点] GenNeighbors --> ForEach[For 每个邻居节点:] ForEach --> InClosed{如果邻居在关闭列表中: 跳过} InClosed -- 跳过 --> CalcF[计算 g(n), h(n), f(n) = g(n) + h(n)] CalcF --> AddOpen[将邻居加入开放列表]</pre>		

↓
结束

2. 算法框图

初始化阶段

- | 1. 创建开放列表(优先队列)和关闭集合
- | 2. 计算起始节点的启发函数值 h (起始状态)
- | 3. 设置起始节点: $g=0$, $f=0+h$, $parent=NULL$
- | 4. 将起始节点加入开放列表



主循环

- | WHILE 开放列表不为空 DO

节点选择与处理

- | 1. 从开放列表取出 f 值最小的节点 $current$
- | 2. IF $current$ 是目标状态 THEN 返回解路径
- | 3. 将 $current$ 加入关闭集合
- | 4. 扩展 $current$ 的所有邻居节点

邻居节点处理

- FOR 每个邻居节点 $neighbor$ DO
 - | | IF $neighbor$ 在关闭集合中 THEN 跳过
 - | | 计算: $g_temp = current.g + 1$
 - | | $h_temp = \text{启发函数}(neighbor)$
 - | | $f_temp = g_temp + h_temp$
 - | | IF $neighbor$ 不在开放列表中 THEN
 - | | 创建新节点并加入开放列表
 - | | ELSE IF $g_temp < neighbor.g$ THEN
 - | | 更新 $neighbor$ 的 g , f 值和父节点



结束处理

- | IF 找到解 THEN
 - | | 重建路径并返回解
- | ELSE
 - | | 返回无解

实验步骤：

1. 理解 A*算法原理和启发式搜索的基本概念
2. 设计八数码问题的状态表示和状态转移规则
3. 实现两种启发式函数：不在位将牌数 $W(n)$ 和曼哈顿距离 $P(n)$
4. 编写 A*算法核心代码，包括开放列表（优先队列）和关闭集合的管理
5. 实现解路径的重建功能，记录每一步的移动方向和状态变化
6. 添加用户交互界面，支持自定义初始状态和目标状态
7. 测试算法正确性，分析不同启发式函数对算法性能的影响
8. 优化代码结构，添加详细的注释和错误处理

实验结果展示及分析：

1. 结果
(1)

请输入初始状态:

请按行输入3x3的数字矩阵:

请输入第1行(3个数字, 用空格分隔, 0代表空格): 2 8 3

请输入第2行(3个数字, 用空格分隔, 0代表空格): 1 6 4

请输入第3行(3个数字, 用空格分隔, 0代表空格): 7 0 5

请输入目标状态:

请按行输入3x3的数字矩阵:

请输入第1行(3个数字, 用空格分隔, 0代表空格): 1 2 3

请输入第2行(3个数字, 用空格分隔, 0代表空格): 8 0 4

请输入第3行(3个数字, 用空格分隔, 0代表空格): 7 6 5

初始状态:

2 8 3

1 6 4

7 0 5

目标状态:

1 2 3

8 0 4

7 6 5

请选择启发函数:

1. 不在位将牌数($W(n)$)

2. 曼哈顿距离($P(n)$)

请输入选择(1或2): 1

使用不在位将牌数($W(n)$)作为启发函数进行求解...

找到解! 总步数: 5

扩展节点数: 6

最大开放列表大小: 8

求解耗时: 0.0000秒

是否显示详细解路径? (y/n): y

解路径:

初始状态:

2 8 3

1 6 4

7 5

步骤1: 移动方向: 上

2 8 3

1 4

7 6 5

步骤2: 移动方向: 上

2 3

1 8 4

7 6 5

步骤3: 移动方向: 左

2 3

1 8 4

7 6 5

步骤4: 移动方向: 下

1 2 3

8 4

7 6 5

步骤5: 移动方向: 右

1 2 3

8 4

7 6 5

(2)

请输入初始状态:

请按行输入3x3的数字矩阵:

请输入第1行(3个数字, 用空格分隔, 0代表空格): 2 8 3

请输入第2行(3个数字, 用空格分隔, 0代表空格): 1 6 4

请输入第3行(3个数字, 用空格分隔, 0代表空格): 7 0 5

请输入目标状态:

请按行输入3x3的数字矩阵:

请输入第1行(3个数字, 用空格分隔, 0代表空格): 1 2 3

请输入第2行(3个数字, 用空格分隔, 0代表空格): 8 0 4

请输入第3行(3个数字, 用空格分隔, 0代表空格): 7 6 5

初始状态:

2 8 3

1 6 4

7 0 5

目标状态:

1 2 3

8 0 4

7 6 5

请选择启发函数:

1. 不在位将牌数($W(n)$)

2. 曼哈顿距离($P(n)$)

请输入选择(1或2): 2

使用曼哈顿距离($P(n)$)作为启发函数进行求解...

找到解! 总步数: 5

扩展节点数: 5

最大开放列表大小: 7

求解耗时: 0.0000秒

是否显示详细解路径? (y/n): y

解路径:

初始状态:

2 8 3

1 6 4

7 5

步骤1: 移动方向: 上

2 8 3

1 4

7 6 5

步骤2: 移动方向: 上

2 3

1 8 4

7 6 5

步骤3: 移动方向: 左

2 3

1 8 4

7 6 5

步骤4: 移动方向: 下

1 2 3

8 4

7 6 5

步骤5: 移动方向: 右

1 2 3

8 4

7 6 5

2. 估价函数的值对搜索算法速度的影响。

(1) 不在位将牌数 ($W(n)$)

优点: 计算简单, 速度快

缺点: 启发信息较弱, 可能扩展较多节点

效率: 在简单情况下表现良好, 但在复杂情况下效率较低

(2) 曼哈顿距离 ($P(n)$)

优点: 提供更强的启发信息, 能更准确估计到目标的距离

缺点: 计算量稍大

效率: 在大多数情况下比 $W(n)$ 更高效, 扩展节点数更少

实验数据对比

从代码运行结果可以看出：

- 使用 $W(n)$ 时扩展节点数：6
- 使用 $P(n)$ 时扩展节点数：5
- $P(n)$ 相比 $W(n)$ 减少了节点扩展

3. 根据启发式搜索算法分析启发式搜索的特点。

(1) 优点：

高效性：通过启发函数引导搜索方向，避免盲目搜索

最优性：在满足 $h(n) \leq h^*(n)$ 条件下能找到最优解

灵活性：可根据问题特点设计不同的启发函数

(2) 缺点：

启发函数设计困难：好的启发函数需要深入理解问题特性

内存消耗：需要维护开放列表和关闭列表

启发函数质量影响大：差的启发函数可能导致性能下降

(3) 适用场景：

- a) 状态空间较大的问题
- b) 能够设计出有效启发函数的问题
- c) 对解的质量有要求的问题

附件：关键程序代码